

Министерство науки и образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы
алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнил:

студент группы 25BBB1

Цеплов Е.В.

Разживалов Н.А.

Приняли:

к.т.н., доцент Юрова О.В.,

к.т.н., доцент Деев М.В.

Пенза 2026

Название

Бинарное дерево поиска

Цель работы

Изучить бинарное дерево поиска

Лабораторное задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчета числа вхождений заданного элемента в дерево.
3. Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. Оценить сложность процедуры поиска по значению в бинарном дереве.

Ход работы

Задание 1:

Реализован алгоритм поиска:

Функция Поиск(r, target):

Если r = NULL:

Вернуть 0 // не найдено

Если r.данные = target:

Вернуть 1 // найдено

Если target > r.данные:

Вернуть Поиск(r.левый, target) // идём влево

Иначе:

Вернуть Поиск(r.правый, target) // идём вправо

Задание 2:

Реализована функция подсчета числа вхождений заданного элемента в дерево:

Функция Count(r, target):

Если r = NULL:

Вернуть 0

count = 0

Если $r.\text{данные} = \text{target}$:

count = 1

count = count + Count(r.левый, target)

count = count + Count(r.правый, target)

Вернуть count

Задание 3:

Что бы поменять алгоритм добавления элементов с исключением повторяющихся. Нужно проверять в начале не равна ли информация текущей информации на пути. Поменять else на else if($\text{data} < r \rightarrow \text{data}$), чтобы при равенстве ничего не происходило.

```
/* if (root != NULL && root->data == data)
return root; */ //Исключение повторов
p = (struct Node*)malloc(sizeof(struct Node));
```

Задание 4:

Сложность поиска в бинарном дереве — $O(h)$, где h — высота дерева. В лучшем и среднем случае (сбалансированное дерево) это $O(\log n)$, в худшем (вырожденное дерево) — $O(n)$. Поскольку в нашей реализации балансировка не выполняется, сложность зависит от порядка ввода элементов.

Результат работы программы

Задание 1:

```
Построение дерева окончено
1
 2
  3
   4
    5
     6
      7
       8
        9
         10

Выберите вариант работы с деревом
1 – Поиск значения
2 – Подсчёт вхождений
3 – Выход
Ваш выбор: 1
Введите число для поиска: 5
Элемент 5 найден в дереве.
```

Задание 2:

```
Введите число для подсчёта вхождений: 5
Число 5 встречается 1 раз(a).
```

```
Выберите вариант работы с деревом
```

```
1 – Поиск значения
```

```
2 – Подсчёт вхождений
```

```
3 – Выход
```

```
Ваш выбор: |
```

Задание 3:

```
0 – окончание построения дерева
```

```
Введите число: 1
```

```
Введите число: 1
```

```
Введите число: 1
```

```
Введите число: 1
```

```
Введите число: 2
```

```
Введите число: 2
```

```
Введите число: 2
```

```
Введите число: 2
```

```
Введите число: 4
```

```
Введите число: 4
```

```
Введите число: 4
```

```
Введите число: 5
```

```
Введите число: 5
```

```
Введите число: 5
```

```
Введите число: 0
```

```
Построение дерева окончено
```

```
1
```

```
  2
```

```
    4
```

```
      5
```

```
Выберите вариант работы с деревом
```

```
1 – Поиск значения
```

```
2 – Подсчёт вхождений
```

```
3 – Выход
```

```
Ваш выбор: |
```

Вывод

В лабораторной работе реализованы основные операции над бинарным деревом поиска: поиск элемента, подсчёт числа вхождений и запрет добавления дубликатов. Показано, что сложность поиска зависит от высоты дерева: в среднем — $O(\log n)$, в худшем (вырожденном) — $O(n)$. Без балансировки эффективность дерева зависит от порядка ввода элементов.